

Deskwork. Set the paper, check the papers, clear the desk

One place for the work teachers still do by hand. Check handwritten answer booklets against an approved rubric, with evidence for every mark and the teacher in charge. Generate assignments and papers from the course's own slides. Ask a built-in assistant for help anywhere in the app. More tools follow.

Users Teachers only, any college **Milestones** Mid eval: working prototype · Final eval: finished product
AI Gemini 3.5 Flash-Lite + 3.1 Flash-Lite, free tier **Demo subject** Whichever subject's material you share

§1 Verdict

Two research agents reviewed the plan independently. One looked at it from the teacher, product and institution side. The other looked at the engineering and the Gemini API. Their joint conclusion, which I agree with:

The idea is good, as long as the pitch stays honest. "One AI grades everyone, so it is fair" won't survive a viva. Current research shows vision models change 50 to 64% of criterion marks when the same answer is graded again. They also quietly "fix" student mistakes while reading the handwriting.

The version you can defend: two independent models grade each booklet directly from the scanned image. Every mark cites visible evidence. When the models disagree, the answer goes to the teacher, and the UI makes sure the teacher actually reviews it. Around that sit an assignment generator grounded in the teacher's own slides and an assistant available everywhere. Pitch: *"Deskwork does the repetitive work: checking, setting papers, drafting. The teacher stays the examiner and the author."*

§2 What the critique changed

Each serious criticism, and the decision it led to. These decisions are already applied throughout the plan below.

Criticism	Evidence	Decision
Google may use free-tier Gemini inputs for training and have humans review them. Student scripts are personal data under India's DPDP Act 2023.	Gemini API terms; DPDP guidance	The demo uses your own handwritten answers, which you've consented to. Names are masked locally before anything is uploaded, and students get pseudonymous IDs. A config switch moves real use to a paid tier that doesn't train on data. Each exam has a delete button.
"Teacher has final say" is weak on its own: teachers who were told a grade came from AI corrected wrong harsh grades 22% less often	PNAS Nexus 2026, 1,300+ teachers	Forced review UX: the AI mark stays hidden until the scan has been viewed. Accepting a harsh mark needs a one-tap reason. A random 10% spot-check comes before bulk accept.
Transcribing first and then grading hides errors: models quietly correct 42 to 66% of student mistakes	arXiv 2604.22774	Grade from the image . The transcript is evidence for the teacher, never the grading input.
The original call plan needs about 612 calls for 60 students, more than the ~500 per day limit	Quota math, \$15	One call per booklet per model , about 60 per model. Targeted re-grades only for flagged answers.
Temperature 0 degrades Gemini 3 models (they can get stuck looping)	Google's Gemini 3 guide	Leave temperature at the default. Stability comes from a strict schema, <code>thinking_level</code> , and agreement between two models .
Model confidence alone gives "no stable threshold"	arXiv 2603.13083	Flag on several signals: model disagreement, low confidence, answer type, cross-page answers, crossed-out work, numeric mismatch, harsh marks.
Handwritten code and diagrams grade poorly (QWK 0.09 on handwritten code)	arXiv 2609.14284, 2608.20521	These are graded like everything else but always flagged for teacher review, presented as a deliberate design choice.
Coarse 0 to 5 criteria double the error	Same studies	Each rubric criterion is worth 1 to 3 marks, in steps of 0.5, with accepted alternatives and common errors listed.

Criticism	Evidence	Decision
Mapping pages to questions is the hardest part: answers are out of order, and students add supplementary sheets	Gradescope needs fixed templates for this	The model returns a page map, and the teacher can reassign pages. Page-count and blur checks run at upload.
The CO printed on a paper doesn't reliably match the question	Your input	The printed CO is stored as-is for reference and never used for any logic.
CBSE's 2026 on-screen marking scandal: blurred scans, missing pages, mixed-up sheets	Widely reported	Capture checks: expected vs actual page count, blur score, and booklet ID on every page. Nothing is published automatically.
Too much scope means every feature gets a shallow demo	Both agents	Mid eval builds three things properly: checking, generator, basic assistant. Everything else appears in the UI as Coming soon (\$14).
Hosting: Render's free tier has no background workers, Vercel caps functions at 300 s, and free Supabase projects pause after a week idle	Provider docs	Next.js on Vercel, Supabase, and one Python worker for long jobs (a laptop for mid, Cloud Run for final). A daily ping keeps Supabase awake.
Two backends (FastAPI plus Next.js) is too much for a small team	Engineering review	No REST API in Python. The web app talks to Supabase directly; Python only runs jobs from a queue table. Assistant chat runs in one Next.js route.

§3 Product definition

WHO

- Teachers only, at any college. The demo uses JIIT material.
- Students never log in. Papers and results still reach them through Google Classroom or WhatsApp.

PROBLEM

- Teachers still do checking, paper setting and drafting by hand.
- Checking 60 to 200 booklets takes days, different checkers mark differently, and students dispute marks.
- Writing fresh assignments and papers from slides eats hours every week.

Modules

Module	Mid eval	Final eval
Exam checking (headline feature)	LIVE full flow	Deeper grading, faster review
Assignment and paper generator	LIVE basic	Sets, Word export, per-question edits
Assistant (chatbot on every page)	LIVE basic tools	Multi-step actions with confirmation
Lesson planner, question bank, feedback writer, rubric library, analytics	COMING SOON visible in the UI	Built as time allows

What makes it different

- **Two-model agreement** instead of trusting one model, with every mark citing the student's own words.
- **Designed against automation bias:** the UI makes the teacher actually look before accepting.
- **Grounded in the teacher's material:** the generator and assistant work from the course's own slides, with page references.

- **Built for Indian exams:** T1, T2 and End Sem booklets, half marks, supplementary sheets.
- **Free to run** on the Gemini free tier for a class-sized batch.

JIIT exam scheme used as the default preset

Taken from the JIIT B.Tech Academic System document (revised Sep 2025). It sets the default exam types and max marks, and other colleges can edit it.

Component	Marks	Duration	Syllabus
T1	20	1 hour	Up to T1
T2	20	1 hour	T1 to T2
End Semester	35	2 hours	Full syllabus
Teacher Assessment (TA)	25	Continuous	Assignments, quizzes, etc.

Marked scripts are shown to students, so an evidence view for each question directly helps on script-showing day. Grading is relative, so the results screen shows the marks distribution but does not assign grades.

§4 Scope: mid vs final

Rule: at mid eval, each live flow works end to end on real handwritten scripts.

MID features are detailed in §12. **FINAL** features are scoped briefly in §19 and appear in the prototype as **COMING SOON**.

Area	Mid eval (build fully)	Final eval (scope only)
Accounts and courses	Email login, courses, student CSV, slide upload with text extraction by unit	TA access, exam scheme presets for other colleges, course archive
Exam setup	Upload the question paper; AI extracts questions, marks, answer type and Bloom level; teacher edits	Question bank reuse across exams
Answer key and rubric	Typed or handwritten key becomes a draft rubric; teacher approves a frozen version	Rubric edits re-apply to already graded scripts
Script capture	PDF or image upload per booklet, page rendering, local name mask, page-count and blur checks	Phone batch capture with deskew, QR cover sheet, drag to fix page order
Grading	One call per booklet on two models, all answer types, criterion scores with evidence, flags, numeric tolerance check, cache, throttle	Targeted per-question re-grade, SymPy step checks, test runs for typed code, diagram label checklists
Review	Flag queue, scan and evidence side by side, forced review, edit with reason, audit log, keyboard shortcuts	Question-wise grading across all booklets with answer grouping, script-showing-day view
Results	Marks table, marks distribution, per-question stats, Excel export	Weak-topic analytics, comparison across exams
Generator	Pick units, question types, count and marks; get questions, answer key, rubric and Bloom level; edit; printable PDF; convert to an exam in one click	Sets A/B/C, college-format Word file, regenerate a single question, chat tweaks
Assistant	Chat panel on every page: answers from slides with page citations, questions about results, drafts short text, opens pages, pre-fills the generator	Multi-step actions with confirmation (create exam, run generator, bulk operations), explains a student's marks, remembers preferences
Coming-soon placeholders	Greyed-out cards and buttons where future features will live (\$14)	Each one becomes live when built by flipping a flag
Evaluation	Harness on your answers plus team-written booklets with planted errors	A study of 50+ booklets across all four answer types

§5 Test data plan

You'll provide one of each item below for one subject. The handwritten answers are your own, with full consent, and they're also what the viva demo uses. Each item has a defined job in the pipeline and in the evaluation.

You provide	Used for	Pipeline step
Previous-year question paper (PDF)	Question extraction: labels, sub-parts, marks, type	C1 extract_paper
Typed answer key for that paper	Draft rubric from a typed key	C2 build_rubric (typed)
Handwritten answer key example	Draft rubric from handwriting (image input)	C2 build_rubric (handwritten)
Subject PDF slides	Text by unit; grounding for the generator and the assistant	extract_material, C5, C6
A college assignment	Second exam type (kind = ASSIGNMENT); style reference for the generator	C1, C5 style sample
Your handwritten answers	End-to-end grading test and the live demo	ingest_scripts, C3/C4 grade_booklet

Make it a real test set

- **Human marks:** have a teacher (or a teammate, using the approved rubric) mark your answers. Without human marks there is no accuracy number.
- **More booklets:** each team member hand-writes 3 to 5 answer booklets for the same paper, with **deliberately planted errors** recorded in a sheet. Plant a wrong step, a wrong unit, a missing definition, a crossed-out attempt, and an answer continued on a later page. Aim for 12 to 20 booklets.
- **Human baseline:** two team members mark the extra booklets blind with the approved rubric. The AI-vs-human gap is then compared with the human-vs-human gap.
- **Cover all four types:** include at least one theory, one numerical, one code and one diagram question. If the provided paper lacks a type, add a question to the extra booklets.

```

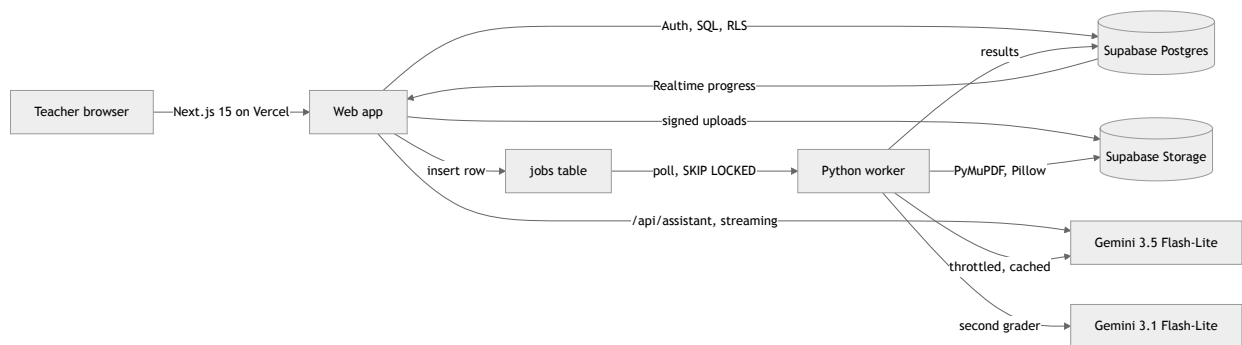
testdata/
  subject-xyz/
    paper_pyq.pdf
    key_typed.pdf
    key_handwritten.pdf
    slides/unit1.pdf ... unitN.pdf
    assignment_1.pdf
  scripts/                                # one PDF per booklet, file name = anon id
    S001.pdf  S002.pdf ...
  labels/
    human_marks.csv                      # anon_id, question, criterion, marks, marker
    planted_errors.csv                   # anon_id, question, error_type, note
    transcripts_gold/                    # 5 pages typed by hand, for CER

```

Keep scripts out of git (`.gitignore`) and out of any public repo.

§6 Architecture

One web app, one database, one Python worker. The browser never calls Gemini directly. Long jobs run in the worker, and assistant chat runs in one server-side Next.js route.



Layer	Choice	Reason
Web	Next.js 15 App Router, TypeScript, Tailwind, shadcn/ui, TanStack Table, Recharts, @google/genai (assistant route only)	The team already knows it; shadcn gives a clean, consistent dashboard
Data	Supabase: Postgres with RLS, Auth (email), Storage, Realtime, full-text search	One service covers auth, data, files, live progress and slide search
Worker	Python 3.12, google-genai , PyMuPDF, Pillow, OpenCV (blur check), Pydantic, openpyxl, psycpg	The PDF, image and numeric libraries are best in Python
Queue	jobs table with FOR UPDATE SKIP LOCKED	No extra infrastructure, and jobs can be restarted
Hosting	Vercel (web), Supabase free tier; worker on an always-on laptop for mid and a Docker image on Cloud Run for final	Free options that support long-running jobs
Feature flags	lib/features.ts : one list of every feature and its state (live or soon)	Drives every coming-soon placeholder from one file

§7 Repo layout

```
deskwork/
  apps/web/                                Next.js app
    app/(auth)/login/
    app/(app)/dashboard/
    app/(app)/courses/[courseId]/          exams, materials, students tabs
    app/(app)/courses/[courseId]/exams/new/ setup wizard
    app/(app)/exams/[examId]/              overview + progress
    app/(app)/exams/[examId]/rubric/
    app/(app)/exams/[examId]/scripts/
    app/(app)/exams/[examId]/review/
    app/(app)/exams/[examId]/results/
    app/(app)/courses/[courseId]/generate/
    app/(app)/soon/[feature]/              shared coming-soon page for sidebar i
    app/api/assistant/route.ts             streaming chat + tool calls
  lib/supabase/ lib/jobs.ts lib/features.ts lib/assistant/tools.ts
  lib/types.ts                            generated from the DB
  components/ui/ (shadcn) components/review/ components/rubric/
  components/assistant/ components/coming-soon.tsx
worker/
  deskwork_worker/
    main.py                                job loop
    jobs/extract_material.py extract_paper.py build_rubric.py
    jobs/ingest_scripts.py grade_booklet.py finalize_flags.py
    jobs/generate_set.py export_results.py
    ai/client.py                           throttle, retry, fallback, cache, usage
    ai/schemas.py                         Pydantic models = JSON schemas
    ai/prompts/*.md                        versioned prompt files
    pdf.py mask.py quality.py numeric.py
  pyproject.toml Dockerfile
  supabase/migrations/0001_init.sql
  eval/harness.py eval/metrics.py eval/reports/
  testdata/ (gitignored)
  .env.example                            GEMINI_API_KEY, SUPABASE_URL, SUPABASE_SERVICE_RO
```

§8 Database schema

The full schema for the mid milestone. Every table holding course data gets an RLS policy that lets only the course owner see it. The worker uses the service role key.

```

-- supabase/migrations/0001_init.sql
create extension if not exists pgcrypto;

create table profiles (
  id uuid primary key references auth.users on delete cascade,
  full_name text, institution text, created_at timestamptz default now());

create table exam_schemes (
  id uuid primary key default gen_random_uuid(),
  owner_id uuid references profiles, name text not null,
  components jsonb not null);
-- [{"kind":"T1","max":20},...]

create table courses (
  id uuid primary key default gen_random_uuid(),
  owner_id uuid not null references profiles on delete cascade,
  code text not null, name text not null, semester text, section text,
  scheme_id uuid references exam_schemes,
  created_at timestamptz default now());

create table materials (
  id uuid primary key default gen_random_uuid(),
  course_id uuid not null references courses on delete cascade,
  kind text not null check (kind in ('slides','notes','assignment','pyq','other'))
  title text not null, unit text, storage_path text not null,
  status text not null default 'uploaded', -- uploaded|extracted|error
  created_at timestamptz default now());

create table material_pages (
  id uuid primary key default gen_random_uuid(),
  material_id uuid not null references materials on delete cascade,
  page_no int not null, text text, topics text[],
  tsv tsvector generated always as (to_tsvector('english', coalesce(text,''))) sto
  unique (material_id, page_no));
create index on material_pages using gin (tsv); -- assistant slide search

create table students (
  id uuid primary key default gen_random_uuid(),
  course_id uuid not null references courses on delete cascade,
  anon_id text not null, -- S001, shown everywhere by default
  roll_no text, name text, -- stay in our DB, never sent to AI
  unique (course_id, anon_id));

create table exams (
  id uuid primary key default gen_random_uuid(),
  course_id uuid not null references courses on delete cascade,
  title text not null,
  kind text not null check (kind in ('T1','T2','END_SEM','ASSIGNMENT','QUIZ','OTHE
  max_marks numeric, paper_path text,
  mask_region jsonb, -- {"page":1,"x":..,"y":..,"w":..,"h"

```

```

    expected_pages int,
    status text not null default 'draft',      -- draft|rubric_ready|grading|review|
    created_at timestamptz default now());

create table questions (
    id uuid primary key default gen_random_uuid(),
    exam_id uuid not null references exams on delete cascade,
    label text not null, ord int not null,      -- '1', '2a', '2b'
    text text not null, max_marks numeric not null,
    answer_type text not null check (answer_type in ('theory','numeric','code','diag
    bloom text check (bloom in ('remember','understand','apply','analyze','evaluate'
    printed_co text,                          -- as printed on the paper; reference
    unique (exam_id, label));

create table rubric_versions (
    id uuid primary key default gen_random_uuid(),
    exam_id uuid not null references exams on delete cascade,
    version int not null,
    status text not null default 'draft' check (status in ('draft','approved','super
    source text not null check (source in ('typed_key','handwritten_key','manual','g
    key_path text, approved_at timestamptz,
    unique (exam_id, version));

create table rubric_items (
    id uuid primary key default gen_random_uuid(),
    rubric_version_id uuid not null references rubric_versions on delete cascade,
    question_id uuid not null references questions on delete cascade,
    criterion_key text not null,                -- 'q2a.c1'
    description text not null,
    marks numeric not null check (marks > 0 and marks ≤ 3 and marks * 2 = floor(mar
    accept_alternatives text[] default '{}', common_errors text[] default '{}',
    final_answer text, tolerance_pct numeric, units text, ord int not null);

create table submissions (
    id uuid primary key default gen_random_uuid(),
    exam_id uuid not null references exams on delete cascade,
    student_id uuid references students,
    storage_path text not null, page_count int,
    quality jsonb,                            -- {"page_count_ok":true,"blurry_page
    status text not null default 'uploaded',    -- uploaded|ready|grading|graded|revi
    created_at timestamptz default now());

create table submission_pages (
    id uuid primary key default gen_random_uuid(),
    submission_id uuid not null references submissions on delete cascade,
    page_no int not null, image_path text not null,
    blur_score numeric, masked boolean default false,
    unique (submission_id, page_no));

create table grading_runs (

```

```

id uuid primary key default gen_random_uuid(),
submission_id uuid not null references submissions on delete cascade,
rubric_version_id uuid not null references rubric_versions,
grader text not null check (grader in ('A','B','regrade')),
model text not null, prompt_version text not null, cache_key text not null,
raw jsonb, input_tokens int, output_tokens int, latency_ms int,
status text not null default 'ok', error text, created_at timestamptz default no

create table answer_grades (
    id uuid primary key default gen_random_uuid(),
    run_id uuid not null references grading_runs on delete cascade,
    question_id uuid not null references questions,
    pages int[], transcript text, final_answer text,
    flags text[] default '{}', confidence numeric, total numeric);

create table criterion_scores (
    id uuid primary key default gen_random_uuid(),
    answer_grade_id uuid not null references answer_grades on delete cascade,
    rubric_item_id uuid not null references rubric_items,
    points numeric not null, evidence text);

create table final_marks (
    id uuid primary key default gen_random_uuid(),
    submission_id uuid not null references submissions on delete cascade,
    question_id uuid not null references questions,
    ai_marks numeric,
    marks numeric,
    criteria jsonb,
    review_status text not null default 'pending'
    check (review_status in ('pending','flagged','accepted','edited','spot_checked')),
    flag_reasons text[] default '{}',
    reviewed_by uuid references profiles, reviewed_at timestamptz,
    unique (submission_id, question_id));

create table audit_log (
    id bigserial primary key,
    course_id uuid references courses, entity text not null, entity_id uuid not null,
    action text not null, before jsonb, after jsonb, reason text,
    actor uuid, at timestamptz default now());

create table generated_sets (
    id uuid primary key default gen_random_uuid(),
    course_id uuid not null references courses on delete cascade,
    title text, config jsonb not null, content jsonb, status text default 'queued',
    created_at timestamptz default now());

create table chat_threads (
    id uuid primary key default gen_random_uuid(),
    owner_id uuid not null references profiles on delete cascade,
    course_id uuid references courses on delete set null,

```

```

    title text, created_at timestampz default now());

create table chat_messages (
    id bigserial primary key,
    thread_id uuid not null references chat_threads on delete cascade,
    role text not null check (role in ('user','assistant','tool')),
    content jsonb not null,                -- text, tool calls, tool results, ci
    created_at timestampz default now());

create table jobs (
    id bigserial primary key,
    kind text not null, payload jsonb not null,
    status text not null default 'queued' check (status in ('queued','running','done
    attempts int not null default 0, progress jsonb, error text,
    run_after timestampz default now(), locked_at timestampz,
    created_at timestampz default now());
create index on jobs (status, run_after);

create table ai_cache (
    cache_key text primary key,            -- sha256(model|prompt_version|rubric
    model text, response jsonb not null, created_at timestampz default now());

create table ai_usage_daily (
    day date not null, model text not null, source text not null,    -- worker|assist
    calls int default 0, tokens int default 0,
    primary key (day, model, source));

```

§9 AI calls and prompts

There are six call types. C1 to C5 run in the worker through `ai/client.py`, which throttles, retries, falls back to the other model, caches, and logs usage. C6 is the assistant route. Prompts live in versioned files, so the cache key changes whenever a prompt changes.

Call	Input	Output	Model / thinking	Calls
C1 extract_paper	Question paper PDF	Questions: label, text, marks, type, Bloom, printed CO (copied as-is)	3.5 / low	1 per exam
C2 build_rubric	Questions plus typed key text <i>or</i> handwritten key page images	Criteria per question (0.5 to 3 marks each), final answers, alternatives, and the key transcript	3.5 / medium	1 per exam
C3 grade_booklet (A)	Rubric JSON plus every page image of one booklet	Page map, per- question transcript, criterion points with evidence, flags, confidence	3.5 / medium	1 per booklet
C4 grade_booklet (B)	Same	Same	3.1 / medium	1 per booklet
C5 generate_set	Full text of the selected units, config, style sample	Questions plus answer key, rubric, Bloom level and source pages	3.5 / medium	1 per set
C6 assistant	Conversation, page context, tool results	Streamed reply, tool calls	3.5 / low	1 to 3 per message

Model settings (checked against the Gemini docs by the research agent)

- Model IDs: `gemini-3.5-flash-lite` and `gemini-3.1-flash-lite`. Both take 1M input tokens and return up to 64k.
- **Do not set temperature, top_p or top_k.** Google recommends the defaults for Gemini 3; a low temperature can make the model loop.
- `thinking_level`: `minimal` is the Flash-Lite default. Use `low` for extraction and the assistant, and `medium` for grading and rubrics. Pick the final values with the harness.

- `media_resolution` : high (about 1,120 tokens per image) for handwriting. Try `medium` in the harness to save tokens.
- Use structured output with a JSON Schema generated from Pydantic, and keep schemas shallow.
- Don't store slides in the Files API, because files expire after 48 hours. Context caching isn't on the free tier either, so send extracted text instead.

```
# worker/deskwork_worker/ai/client.py (sketch; confirm field names against current
from google import genai
from google.genai import types

client = genai.Client() # reads GEMINI_API_KEY

def call_json(model, system, parts, schema_model, thinking="medium", media="high")
    key = cache_key(model, system, parts, schema_model)
    if (hit := cache_get(key)): return hit
    bucket.acquire(model, est_tokens(parts)) # RPM + TPM token bucket per
    resp = with_retry(lambda: client.models.generate_content(
        model=model,
        contents=parts,
        config=types.GenerateContentConfig(
            system_instruction=system,
            response_mime_type="application/json",
            response_json_schema=schema_model.model_json_schema(),
            thinking_config=types.ThinkingConfig(thinking_level=thinking),
            media_resolution=f"MEDIA_RESOLUTION_{media.upper()}",
        ))) # retry 429/503: exp backoff
    data = schema_model.model_validate_json(resp.text) # one re-ask on validation
    cache_put(key, model, data); usage_inc(model, "worker", resp.usage_metadata)
    return data

# generate_content is the long-supported path; the newer Interactions API also works
# If using Interactions, check whether it stores data server-side by default and t
```

Schemas (Pydantic models, converted to JSON schema)

```
class RubricCriterion(BaseModel):
    key: str # "q2a.c1"
    description: str
    marks: float # 0.5 to 3, steps of 0.5
    accept_alternatives: list[str] = []
    common_errors: list[str] = []

class RubricQuestion(BaseModel):
    label: str
    criteria: list[RubricCriterion]
    final_answer: str | None = None
    tolerance_pct: float | None = None
    units: str | None = None

class RubricDraft(BaseModel):
    key_transcript: str # what the model read from the key (teacher check)
    questions: list[RubricQuestion]

class PageInfo(BaseModel):
    page: int
    questions: list[str]
    legible: Literal["good", "partial", "poor"]

class CriterionScore(BaseModel):
    key: str
    evidence: str # quote or describe the student's own work FIRST
    points: float # clamped in code to [0, marks], rounded to 0.5

Flag = Literal["cross_page", "crossed_out", "diagram", "code", "illegible",
               "unanswered", "alt_method", "possible_misread"]

class AnswerGrade(BaseModel):
    label: str
    pages: list[int]
    transcript: str # verbatim, keep errors, [crossed], [illegible]
    final_answer: str | None = None
    criteria: list[CriterionScore]
    flags: list[Flag] = []
    confidence: float # 0 to 1

class BookletGrade(BaseModel):
    pages: list[PageInfo]
    answers: list[AnswerGrade]
```

Grading system prompt (prompts/grade_booklet.v1.md)

You are an exam checker applying an approved rubric to one student's scanned answers. Pages are labelled "PAGE n". The student is anonymous.

Rules:

1. Grade only what is visibly written. Never assume what the student meant.
2. Do NOT correct the student's errors, even small ones, in the transcript or in your answer. A wrong sign, wrong unit or wrong step stays wrong.
3. Ignore handwriting neatness, grammar, spelling and answer length. Judge content.
4. Crossed-out work is withdrawn and earns nothing, unless nothing else is written.
5. Answers may be out of order or continue on later pages. Use question labels written by the student to build the page map. If a label is missing, infer from content and add it.
6. For each rubric criterion: first write evidence (quote or describe the student's answer), then award points from 0 to that criterion's marks in steps of 0.5.
7. Accept any method listed in `accept_alternatives`, and any other correct method (if not listed).
8. If a question is not attempted, return it with 0 points and flag "unanswered".
9. Always flag "diagram" for drawings and "code" for program code.
10. `confidence` = how sure you are the awarded points match what an expert examiner would award.

Return JSON matching the schema. Include every question in the rubric.

Parts are sent in this order: the system prompt, then the approved rubric JSON (with question text, max marks and answer type), then each page as the text "PAGE n" followed by its image.

Rules enforced in code, never trusted from the model

- Clamp each criterion to `[0, marks]`, round to 0.5, and recompute the question totals and booklet total.
- Reject criterion keys that aren't in the rubric. A missing criterion counts as 0 and adds a flag.
- Numeric questions: parse `final_answer` and compare it with the key within `tolerance_pct`. If they don't match but the model awarded the final-answer criterion, add the flag `numeric_mismatch`.

§10 Grading pipeline, step by step

Teacher uploads paper



C1 extract questions



Teacher edits questions,
marks, type



Upload typed or
handwritten key



C2 draft rubric + key
transcript



Teacher edits and
approves: frozen rubric
version

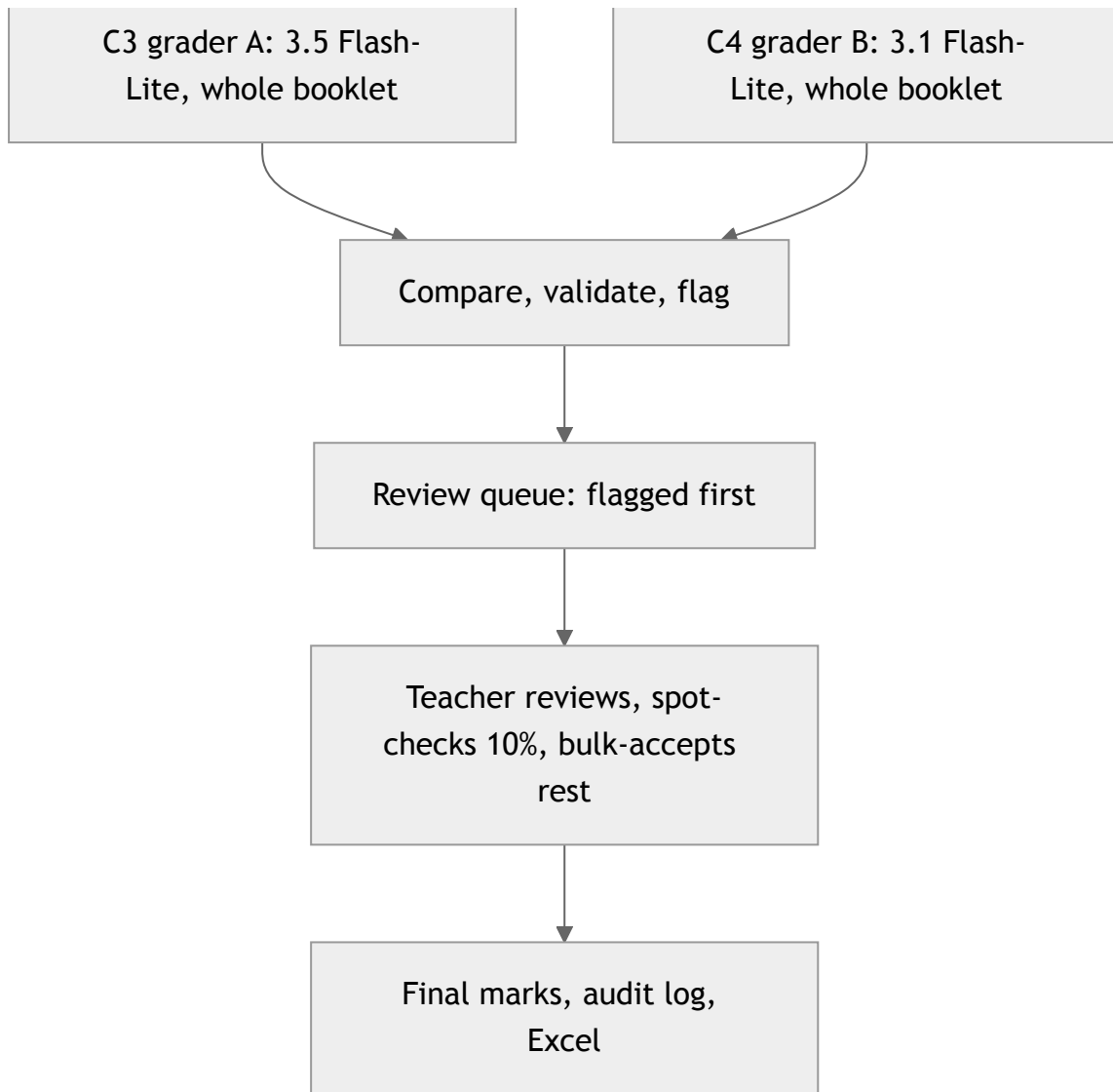


Upload booklets



Render pages, mask name
locally, page count + blur
check





Flag rules (finalize_flags job)

A question in a booklet is **flagged** if any of these is true:

- Graders A and B differ by 1 mark or more on any criterion, or by 1.5 or more on the question total.
- Either grader's confidence is below 0.7.
- The answer type is code or diagram.
- It carries any of these flags: `cross_page`, `crossed_out`, `illegible`, `possible_misread`, `alt_method`, `numeric_mismatch`.
- A page it sits on scored below the blur threshold, or the booklet failed the page-count check.
- The AI awarded less than 40% of max marks. Harsh AI errors are the ones teachers miss most.

For unflagged questions, `ai_marks` is the mean of A and B rounded to 0.5, and the status is `pending`. Flagged questions get the status `flagged`, and both graders' views are stored for the review screen.

Worker loop

```
# worker/deskwork_worker/main.py
CLAIM = """
update jobs set status='running', locked_at=now(), attempts=attempts+1
where id = (select id from jobs where status='queued' and run_after ≤ now()
            order by id for update skip locked limit 1)
returning *;"""
while True:
    job = db.fetchone(CLAIM)
    if not job: sleep(2); continue
    try:
        HANDLERS[job.kind](job.payload, progress=lambda p: set_progress(job.id, p)
        mark_done(job.id)
    except RateLimited as e:
        requeue(job.id, delay=e.retry_after)          # no attempt penalty
    except Exception as e:
        fail_or_retry(job.id, e, max_attempts=3)
# Job kinds: extract_material, extract_paper, build_rubric, ingest_scripts,
#             grade_booklet, finalize_flags, generate_set, export_results
# Every handler is idempotent: re-running it overwrites its own rows.
```

§11 Assistant chatbot

A chat panel that opens from any page (`Cmd` + `J`, or the button at the bottom right). It knows which course or exam the teacher is looking at. It works through a small set of tools, never by querying the database freely.

How it works

- **Route:** `app/api/assistant/route.ts` streams replies using `@google/genai` with function calling. The Gemini key stays on the server.
- **Context sent with each message:** the current route, the course or exam id and title, and the last 10 messages of the thread.
- **Tools run with the teacher's own Supabase session**, so RLS limits them to that teacher's data.

- **Read-only in the prototype.** Anything that would change data returns a link or a pre-filled form, and the teacher clicks the final button.
- **Limits:** at most 2 tool rounds per message, so each message costs 1 to 3 calls. Usage is logged in `ai_usage_daily` with `source='assistant'`.
- **Citations:** answers drawn from slides show chips like "Unit 2 slides, p. 14" that open the page.
- **Privacy:** results tools return anon IDs only. Names are never sent to the model.

Prototype tools MID

Tool	What it does	Example request
<code>search_materials(course_id, query)</code>	Full-text search over slide pages (Postgres <code>tsvector</code>); returns the top 6 pages' text with references	"Explain 3NF the way my Unit 2 slides do"
<code>get_exam_summary(exam_id)</code>	Average, highest, lowest, distribution, per-question averages, flag counts	"How did the class do in T2?"
<code>query_marks(exam_id, question?, below?, above?, status?)</code>	Filtered marks rows (anon IDs)	"Who scored under 8?", "Which question had the lowest average?"
<code>list_flagged(exam_id)</code>	Flagged answers grouped by reason	"What's left for me to review?"
<code>open_page(target)</code>	Returns a route the UI navigates to after a click	"Take me to the review screen for T2"
<code>prefill_generator(course_id, config)</code>	Opens the generator with the form already filled in; the teacher presses Generate	"Make a 10-mark assignment on Unit 3 with 2 numericals"

With no tool at all, it also drafts short text: a WhatsApp announcement, viva questions on a topic, or a feedback note.

Assistant UI

- A side panel, 400 px wide, that keeps its thread per course. There's a thread list and a "New chat" button.
- When the chat is empty, it shows four suggestion chips that fit the current page. Advanced chips appear greyed out with "(Coming soon)", for example "Create an exam from this paper".
- Tool activity is shown inline ("Searching Unit 2 slides...") before the answer streams in.
- Links and pre-filled forms appear as small action cards inside the chat.

Final version FINAL

Scope only: it takes actions that change data, but only after the teacher confirms a card in the chat. Examples: create an exam from an uploaded paper, run the generator and turn the result into an exam, bulk-accept the reviewed items. It can also explain one student's marks question by question, compare two exams, carry out multi-step requests, and remember the teacher's preferences (difficulty, format).

§12 Mid-eval features in detail

Each feature lists what the teacher does, what the system does, and when it counts as done.

M1 Accounts and course workspace MID

Teacher does	Signs up with email. Creates a course (code, name, semester, section; exam scheme defaults to the IIIT preset). Imports a students CSV (roll_no, name). Uploads slide PDFs and tags each one with a unit.
System does	Generates anon IDs (S001 and so on). <code>extract_material</code> pulls the text of each page with PyMuPDF and stores it with a search index.
Done when	A course shows its materials with page counts and extraction status, plus the student list. RLS stops one teacher from seeing another teacher's course.

M2 Exam setup from the question paper MID

Teacher does	Creates a new exam and picks its kind (T1, T2, End Sem or Assignment). Uploads the question paper PDF.
System does	C1 extracts questions and sub-parts with their marks, answer type and Bloom level. Any CO printed on the paper is copied into <code>printed_co</code> for reference only. The system checks that the marks add up to the exam's maximum.
UI	An editable question table (label, text, marks, type, Bloom), with a warning banner if the marks don't add up.
Done when	Both the provided previous-year paper and the assignment extract correctly after at most small edits.

M3 Answer key to approved rubric MID

Teacher does	Uploads a typed key (PDF or text) or photos of a handwritten key. Reviews the draft rubric, edits criteria, adds alternatives, and clicks Approve.
System does	C2 returns the key transcript and criteria worth 0.5 to 3 marks each, and checks that each question's criteria add up to its marks. Approving freezes a new <code>rubric_version</code> , and grading always points to a specific version.
UI	A split view for each question: the key (image or text) on the left, and the transcript with the rubric editor on the right. The key transcript is shown so the teacher can catch misreadings of their own handwriting.
Done when	Both the typed key and the handwritten key produce usable rubrics for the test paper.

M4 Script upload and capture checks MID

Teacher does	Drags in booklet PDFs, or one folder of phone photos per booklet. Draws the name and roll-number box once on a sample first page. Matches each file to a student (automatic when files are named by anon ID or roll number).
System does	<code>ingest_scripts</code> renders each page as a grayscale JPEG (about 170 DPI, long side at most 2000 px) and paints over the mask region. It also computes a blur score (variance of Laplacian) and compares the page count with the expected number of pages.
UI	A grid of booklets with thumbnails and status chips such as "Ready", "Blurry page 4" or "9 of 12 pages". A <code>Capture with phone (Coming soon)</code> button sits beside the upload area.
Done when	No image leaves the worker without the mask applied, and bad scans are visible before grading starts.

M5 Dual-model grading MID

Teacher does	Clicks "Check all booklets".
System does	Queues one <code>grade_booklet</code> job per booklet for each grader, and throttles to about 8 calls per minute per model. Results are cached by image hashes, rubric version, prompt version and model. <code>finalize_flags</code> then applies the rules in §10.
UI	Live progress through Supabase Realtime, for example "Checking S014 · 23 of 40 booklets · 6 flagged so far". Never a blank spinner.
Done when	A batch of 40 booklets finishes in one sitting within quota, and re-running it costs zero calls.

M6 Review with forced attention MID

Teacher does	Works through the flagged queue, then spot-checks a random 10% of unflagged answers, then bulk-accepts the rest.
System does	For flagged items, the AI mark stays hidden until the scan region has been on screen, and <code>Space</code> reveals it. Accepting a harsh mark (under 40%) needs a one-tap reason. Bulk accept stays disabled until the spot-check sample is finished. Every change writes to <code>audit_log</code> with the before value, after value and reason.
Shortcuts	<code>Space</code> reveal, <code>A</code> accept, <code>E</code> edit, <code>1</code> to <code>9</code> set criterion points, <code>J</code> / <code>K</code> next/previous, <code>P</code> reassign page.
Coming soon	<code>Question-wise mode</code> <code>Group similar answers</code> sit in the review toolbar.
Done when	A teacher can finish a 40-booklet batch without leaving the keyboard, and every final mark has either a reviewer or an explicit bulk-accept record.

M7 Results and export MID

Teacher does	Opens Results. Switches names on (they're off by default). Exports to Excel.
System does	Shows a marks table by student and question; per-question average, highest mark and % full marks; and a marks histogram to support relative grading. The Excel file has sheet 1 with roll no, name, marks per question and total, and sheet 2 with per-question stats.
Coming soon	<code>Script-showing view</code> <code>Weak topics</code>
Done when	The Excel file opens cleanly and matches the screen.

M8 Assignment and paper generator (basic) MID

Teacher does	Picks units, question types and counts, total marks, difficulty, optional Bloom levels, and an optional style sample (the college assignment). Clicks Generate, then edits any question inline.
System does	C5 runs with the full extracted text of the selected units. Each question comes back with its text, type, marks, Bloom level, answer key, rubric criteria and source page references. The system checks that the marks add up.
Output	A printable question paper page and a separate printable answer key, saved as PDF from the browser. "Use as exam" turns a generated set into an exam in one click, with the rubric included (<code>source='generated'</code>).
Coming soon	Create Set B and C Export as Word Regenerate this question
Done when	Generating from the provided slides produces an assignment a teacher would use after light edits.

M9 Assistant (basic) MID

Teacher does	Opens the panel from any page and asks questions or asks for small tasks.
System does	Everything described in §11: the six read-only tools, slide citations, and results answered with anon IDs.
Done when	It answers a slide question with a correct page citation, answers a results question correctly, and opens the generator already filled in from a single sentence.

M10 Coming-soon placeholders MID

What Greyed-out sidebar items, cards, buttons and assistant chips for final-eval features, all driven from `lib/features.ts`. Full spec in §14.

Done when Every placeholder is clearly inactive, explains what it will do in one line, and becomes live by changing a single flag.

M11 Evaluation harness MID

What `eval/harness.py` runs the pipeline on `testdata/` and writes a report (§16). The team uses it to choose the thinking level, media resolution and prompt version.

Done when The mid-eval slides show real agreement numbers with confidence intervals.

§13 Screens and UI

The look is calm and dense, closer to Linear or Notion than to a colourful school app. Cool neutral greys, one ink-blue accent, and examiner red used only for marks and flags.

Route	Content
/dashboard	A greeting and a "What do you want to do?" row of large cards: Check papers , Generate assignment , Ask assistant , plus greyed <code>Plan a lesson</code> and <code>Write feedback</code> . Below: exams needing review (with flag counts), grading in progress, and recent courses.
/courses/[id]	Tabs: Exams, Materials, Students, and <code>Question bank</code> .
/courses/[id]/exams/new	A stepper: Paper → Questions → Key & rubric → Mask box → Scripts.
/exams/[id]	Status header, a step checklist, live progress, and a summary of flags by reason.
/exams/[id]/rubric	The key on the left and the rubric editor on the right, with an Approve button and version history.
/exams/[id]/scripts	A booklet grid with quality chips. Clicking a booklet opens the page viewer, where pages can be reassigned.
/exams/[id]/review	The key screen. Left 60%: a zoomable scan with this question's pages highlighted. Right 40%: the question, rubric criteria with points steppers, evidence from graders A and B side by side where they differ, flags as chips, and the transcript (collapsed). Top bar: queue position and filters by flag reason. Bottom: shortcut hints.
/exams/[id]/results	Table (TanStack), histogram, per-question stats, and Export.
/courses/[id]/generate	The config form on the left and a live paper preview on the right.
Assistant panel	Slides in from the right on any page (§11).

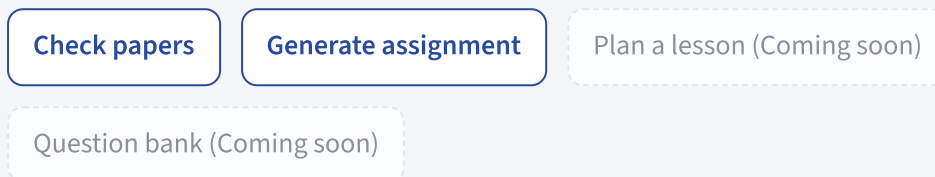
Design tokens for the app

- Fonts: **Geist** or **Source Sans 3** for the UI (not Inter, which is overused), and **JetBrains Mono** for marks and IDs.
- Colours: background #F7F8FA, surface #FFFFFF, text #1A2030, ink-blue accent #2A4A9A, marks red #B8352A, and semantic green and amber for status. Full dark mode.
- Marks appear as circled examiner scores, for example `3 / 4`, a detail taken from real answer booklets.
- Every list has an empty state that points to the next action, and every AI step shows streaming progress text.

- A Cmd+K palette jumps to any course, exam or student, and Cmd+J opens the assistant.

§14 Coming-soon features in the UI

Future features appear exactly where they will eventually live, greyed out and labelled "(Coming soon)". This shows the panel the product vision inside the prototype itself.



One component, one list

```
// lib/features.ts
export const FEATURES = {
  checking:      { status: "live" },
  generator:     { status: "live" },
  assistant:     { status: "live" },
  lessonPlanner: { status: "soon", blurb: "Turn a topic into a lecture plan" },
  questionBank:  { status: "soon", blurb: "Reuse and tag questions across ex" },
  feedbackWriter: { status: "soon", blurb: "Draft personal feedback from mark" },
  rubricLibrary: { status: "soon", blurb: "Save and share rubrics" },
  analytics:     { status: "soon", blurb: "Weak topics and trends across exa" },
  questionWiseReview: { status: "soon", blurb: "Check one question across every b" },
  answerGrouping: { status: "soon", blurb: "Mark similar answers together" },
  phoneCapture:  { status: "soon", blurb: "Scan booklets with your phone cam" },
  qrCoverSheet:  { status: "soon", blurb: "Printable cover sheets for automa" },
  rubricReapply:  { status: "soon", blurb: "Apply rubric changes to already c" },
  multipleSets:  { status: "soon", blurb: "Create Set B and C automatically" },
  wordExport:    { status: "soon", blurb: "Download in your college's paper" },
  regenerateQuestion: { status: "soon", blurb: "Replace a single question" },
  scriptShowingView: { status: "soon", blurb: "A per-question view for showing s" },
  assistantActions: { status: "soon", blurb: "Let the assistant create and upda" },
} as const;

// components/coming-soon.tsx
// <Feature id="multipleSets"><Button>Create Set B and C</Button></Feature>
// live → renders children as-is
// soon → same element, aria-disabled, muted colours, dashed border, "(Coming soo
//        tooltip with the blurb; click opens a small dialog: name, blurb, "Plann
```

Where placeholders go

Place	Placeholders
Sidebar	Lesson planner, Question bank, Feedback writer, Rubric library, Analytics (each opens /soon/[feature] , a clean page with a short description)
Dashboard cards	Plan a lesson, Write feedback
Scripts screen	Capture with phone, Print QR cover sheets
Rubric screen	Re-apply to checked scripts
Review toolbar	Question-wise mode, Group similar answers
Results	Script-showing view, Weak topics
Generator	Create Set B and C, Export as Word, Regenerate this question
Assistant chips	"Create an exam from this paper", "Accept all reviewed answers"

Keep it tasteful. Show at most two placeholders per screen, always next to live features. A screen with more grey than working buttons looks unfinished rather than ambitious. Never let a placeholder look clickable in a way that suggests it works.

§15 Quota budget

Google no longer publishes free-tier numbers in its docs; they only show in AI Studio for each project. Third-party sources say 15 RPM and 500 RPD, and some older ones say 1,000 to 1,500 RPD. **Plan for 250 usable calls per model per day**, and read the real limits in AI Studio before relying on them.

Scenario (60 booklets, 6 questions, ~12 pages)	Model A calls	Model B calls	Fits?
Original plan: page mapping + per-question grading + second pass	~612	0	no
This plan: 1 call per booklet per model	60	60	yes
Plus setup (C1, C2) and one generator run	+3	0	yes
Plus about 30 assistant messages (up to 3 calls each)	+90	0	yes
Final eval: targeted re-grades (at most 20% of answers, split across models)	+36	+36	yes

- Tokens per booklet call: 12 pages × about 1,120, plus about 3k for the rubric, comes to roughly 17k. At 250k TPM that allows about 14 calls a minute, so throttle to 8.
- The assistant falls back to 3.1 Flash-Lite when 3.5 is near its limit.
- Use a separate Google Cloud project (with its own key) for development, so the demo quota is never touched.
- A small settings panel shows `ai_usage_daily`, so the team always knows what's left.

§16 Evaluation harness

METRICS

- Per criterion: exact agreement, agreement within 0.5, MAE
- Per question: MAE, within-1 accuracy, QWK
(`cohen_kappa_score(weights="quadratic")`)
- Per booklet total: MAE, Pearson r, bootstrap 95% CI (1,000 samples)
- Agreement between graders A and B, and the human-vs-human baseline
- Run-to-run stability over 3 runs (% of criteria that change)
- Planted-error detection rate (catches over-correction)
- Flag precision and recall against cases where the AI differs from the human by 1 mark or more
- Transcription character error rate (CER) on 5 hand-typed pages

BREAKDOWNS AND EXPERIMENTS

- Every metric split by answer type: theory, numeric, code, diagram
- Calls, tokens and seconds per booklet
- Experiments: media resolution medium vs high; thinking level low vs medium; with vs without one graded anchor example; 3.5 alone vs A+B agreement
- Reports written to `eval/reports/<date>.md` , with tables ready for the synopsis

Report honestly. A result like "theory answers are within 1 mark 85% of the time, code only 40%, so code always goes to the teacher" is stronger at a viva than one inflated accuracy number.

§17 Privacy and ethics

- **Demo data:** your own handwritten answers, used with your full consent, plus booklets the team writes.
- **Masking:** the worker paints over the name and roll-number box before any AI call. Names never appear in prompts; the AI and the assistant only see S001, S002 and so

on.

- **Free-tier caveat, stated openly:** Google may use free-tier inputs and have humans review them. For real classroom use, the setting `GEMINI_TIER=paid` points to a paid key whose data isn't used for training.
- **Teacher is the examiner:** nothing becomes final or exportable as final until the teacher has reviewed it or bulk-accepted it, with an audit trail.
- **Assistant limits:** read-only in the prototype, scoped by RLS, and it never sees student names.
- **DPDP basics:** a privacy notice at sign-up, data used only for the stated purpose, and a "Delete exam and all scripts" button that removes both the stored files and the database rows.
- **Bias:** the grading prompt tells the model to ignore neatness, grammar and length, and the harness tests this with planted style variations.

§18 Final-eval scope (brief)

Scope only. Each of these is already visible in the prototype as a coming-soon placeholder, and gets detailed planning after mid eval.

GRADING DEPTH

- Targeted re-grade of a single question, using only its pages
- SymPy verification of steps and final answers for numericals
- Typed code: run the teacher's test cases in a sandbox
- Diagram label checklists in the rubric
- Rubric changes re-applied to already checked scripts

REVIEW SPEED

- Question-wise mode: one question across all booklets
- Answer grouping, with one decision for a whole group
- Script-showing view: for each question, the scan, the rubric line, the reason, and the change history
- TA access to a course

CAPTURE

- Phone batch capture with deskew and blur warnings
- Printable QR cover sheets
- Drag to fix page order and add supplementary sheets

GENERATOR AND TEACHER TOOLS

- Sets A/B/C, a Word file in the college format, regenerate one question, chat-based tweaks
- Lesson planner, question bank, feedback writer, rubric library
- Analytics: weak topics, trends across exams

ASSISTANT

- Actions that change data, each confirmed on a card
- Multi-step requests (paper → exam → rubric)
- Explains a student's marks, compares exams, remembers preferences

PLATFORM AND EVIDENCE

- Worker on Cloud Run, Supabase keep-alive, paid-tier switch
- Exam scheme presets for other colleges
- A study of 50+ booklets and a manual-vs-Deskwork time comparison
- A pilot with one or two JIIT teachers

§19 Demo and viva

Mid-eval demo flow

1. The problem in one line: checking takes days, marks are disputed, and paper setting eats hours.
2. The dashboard: three live modules, with coming-soon cards hinting at the rest.
3. Upload the question paper; the questions appear.
4. Upload the **handwritten** key; approve the rubric.
5. Upload booklets and watch dual-model progress live. One booklet is checked live; the rest are already graded.
6. Review: open a flagged answer where the two models disagreed, reveal the marks, fix one with a reason, and show the audit log.
7. Results and the Excel export.

8. Assistant: "Which question did the class struggle with?", then "Make a 10-mark assignment on that topic". The generator opens already filled in.
9. Accuracy numbers by answer type, and what's coming in the final version.

Safety: everything is pre-cached, a screen recording is kept as backup, and the demo uses its own API key.

Questions the panel will ask

Question	Answer
What if the AI is wrong?	It never finalises a mark. Two models have to agree. Disagreements, low confidence, harsh marks, code and diagrams all go to the teacher, and the UI makes sure the teacher really looks. Our measured agreement is X, broken down by answer type.
Is it actually fair?	Every booklet is graded against the same approved rubric, with names hidden. Each mark cites its evidence, and every change is in the audit log. That's more consistent than several different checkers, and fully explainable on script-showing day.
What about student privacy?	Names are masked on our side before the AI sees anything. The demo uses consented data. Real use moves to a paid tier that doesn't train on data. There's a privacy notice and a delete option.
How is this different from Gradescope or MagicSchool?	It combines checking, generation and an assistant in one workspace grounded in the teacher's own slides. It verifies marks with two models, is designed against automation bias, is built for Indian booklets, and is free to run.
What about CBSE's scanning problems?	We check page count and blur at upload and tag every page with its booklet ID. Nothing is published automatically.
Can the assistant break things?	Not in the prototype: it's read-only and limited to the teacher's own data. In the final version, every action needs the teacher to confirm it.
Why teachers only?	The teacher is the accountable examiner, and distribution to students already happens through Classroom and WhatsApp.

§20 Synopsis outline

1. **Title:** Deskwork: An AI Workspace for Teachers with Evidence-Backed Handwritten Exam Evaluation
2. **Abstract** (150 words)
3. **Problem statement:** time spent checking, inconsistency and disputes, repetitive paper setting
4. **Objectives:** rubric-based grading of handwritten scripts with the teacher in the loop; course-grounded assignment generation; an assistant that works inside the app
5. **Literature and existing systems:** Gradescope, Eklavya, E-Valuate, CoGrader, MagicSchool, Brisk; research on vision-LLM grading (over-correction, consistency, automation bias)
6. **Proposed system and architecture:** the §6 diagram and the §10 pipeline
7. **Methodology:** rubric extraction, dual-model grading, flag rules, forced review, the assistant's tool design
8. **Work completed (mid):** M1 to M11, with screenshots
9. **Results so far:** harness tables broken down by answer type
10. **Future scope (final):** the §18 list, matching the coming-soon placeholders in the prototype
11. **Privacy and ethics:** §17
12. **References:** §21

§21 Sources

GEMINI API

- [Gemini 3.5 Flash-Lite](#) · [3.1 Flash-Lite](#) · [Rate limits](#) · [Pricing](#) · [Terms](#)
- [Gemini 3 guide \(temperature\)](#) · [Thinking](#) · [Media resolution](#) · [Structured output](#) · [PDF input](#) · [Files API](#)

RESEARCH ON AI GRADING

- [Vision models over-correct student errors \(2604.22774\)](#)
- [Human-in-the-loop grading of handwritten maths \(2603.13083\)](#)
- [Criterion-level grading \(2609.14284\)](#) · [Physics grading at scale \(2608.20521\)](#)
- [Handwritten maths grading \(2605.19043\)](#) · [Calculus grading \(2510.05162\)](#)

- [Teachers accept harsh AI grading mistakes \(PNAS Nexus coverage\)](#)

INSTITUTION AND MARKET

- [JIIT B.Tech Academic System \(Sep 2025\)](#)
- [DPDP Act for educational institutions](#)
- [2026 CBSE on-screen marking controversy](#)
- [Gradescope answer groups](#) · [Eklavya](#) · [E-Valuate AI](#) · [MagicSchool](#)

HOSTING LIMITS

- [Vercel function duration](#) · [Supabase free pausing](#) · [Render free tier](#) · [Cloud Run pricing](#)

Deskwork build plan v3 · based on two independent research reviews · check the free-tier Gemini limits in AI Studio before relying on them